MERCADO LIEBRE

DOCUMENTO TÉCNICO AVANCE: Plataforma web para publicación y gestión de productos

Materia:

Sistemas Distribuidos

Estudiantes:

Julián Leonardo Cerón
Elkin Yesid Yandun
Brayan Esmid Cruz Chate
Jeison Javier Guerra Enriquez

Docente:
LEIDY MARIANI DORADO FLÓREZ

Facultad:
Ingeniería de sistemas

Corporación Universitaria Comfacauca - Unicomfacauca

Popayán

2026

1. Estado Actual del Proyecto

1.1 Descripción del Avance

Mercado Liebre es una plataforma web que permite a los usuarios registrarse, crear una tienda personalizada (con datos, diseño visual y catálogo de productos) y publicar una vitrina pública accesible por URL. El sistema está completamente contenerizado con Docker Compose, compuesto por tres servicios: el frontend servido por Nginx, la API en Node.js y la base de datos MySQL. El frontend consume la API mediante rutas relativas (/api/...), y es Nginx quien actúa como intermediario (reverse proxy) entre el navegador y el backend.

Componentes implementados

Área	Descripción
Autenticación	Registro e inicio de sesión con email y contraseña. El servidor responde con un token JWT y los datos del usuario.
Backend	Node.js + Express (servicio api-service). API REST con conexión a MySQL mediante pool de conexiones (mysql2).
Base de datos	MySQL 8.0 en contenedor Docker. El esquema se crea automáticamente con init-db/init.sql y los datos persisten en un volumen.
Frontend	React + TypeScript (Vite). Incluye panel de vendedor, gestión de tienda, catálogo de productos y personalización visual.
Proxy HTTP	Nginx redirige las rutas /api/* hacia el servicio api-service:3000. La comunicación entre contenedores usa nombres de servicio, no localhost.
Productos	Endpoints para listar, crear, actualizar y eliminar productos (GET / POST / PATCH / DELETE sobre /api/productos).
Tiendas y temas	Endpoints REST para administrar la tienda del usuario, su tema visual y la vista pública del catálogo.
Imágenes	Subida de imágenes (logo, banner, producto) vía POST /api/media/upload hacia Cloudinary, usando credenciales en el servidor.

Funcionalidades operativas

- Autenticación con email y contraseña; la sesión se mantiene con un token guardado en el navegador (localStorage).
- CRUD completo de productos asociados a la tienda del usuario (escritura requiere autenticación).
- Gestión de datos de la tienda y su tema visual desde el panel.
- Vista pública de la tienda — el endpoint /api/tiendas/:id/vista-publica devuelve tienda, tema y productos en un solo objeto.
- Carga de imágenes cuando las credenciales de Cloudinary están configuradas en el entorno.

Funcionalidades pendientes o planeadas

- Pasarela de pagos.
- Motor de búsqueda y filtrado avanzado en el catálogo público.
- Configuración de seguridad para producción (gestión de secretos, HTTPS, límite de peticiones).

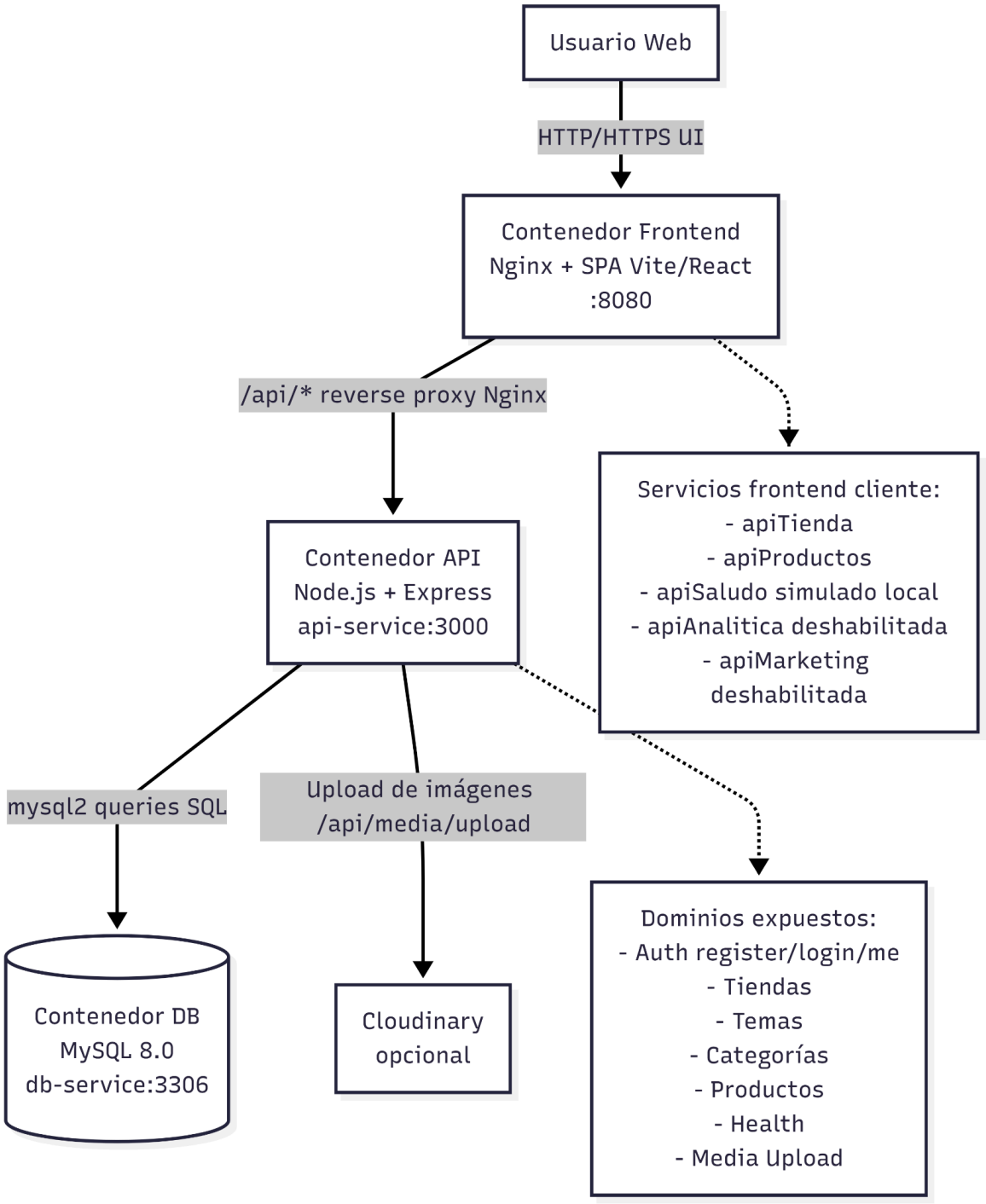
1.2 Tabla de Estado de Componentes

Componente	Estado	Observaciones
Login (POST /api/auth/login)	Implementado	Devuelve token + datos del usuario. Maneja errores 401 / 400 / 500.
Registro (POST /api/auth/register)	Implementado	Campos requeridos: email, password, nombre, apellido.
Sesión (GET /api/auth/me)	Implementado	Requiere cabecera Authorization: Bearer <token>.
API REST (Express)	Implementado	Servicio api-service, puerto 3000 dentro de la red Docker.
MySQL	Implementado	Imagen mysql:8.0, volumen mysqldata, base de datos mercadoliebre_db_prod.
Frontend Nginx	Implementado	Sirve la SPA y redirige /api/ al backend.
Tabla usuarios	Implementado	id (UUID), email, nombre, apellido, password_hash, creado_en.
Tabla tiendas	Implementado	Datos comerciales + URLs de logo y banner.
Tabla temas	Implementado	Colores, fuentes y estilo de plantilla por tienda_id.
Tabla categorías	Implementado	Asociada a tienda_id; usada en el modelo de datos.
Tabla productos	Implementado	No existe tabla artículos — la entidad correcta es productos.
GET /api/productos	Implementado	Admite filtros opcionales ?tienda_id= y ?activo=.
POST /api/media/upload	Implementado	Requiere JWT y variables de Cloudinary configuradas en el API.
Docker Compose (3 servicios)	Implementado	frontend, api-service, db-service en red mercadoliebre-net.
Pasarela de pagos	Pendiente	No desarrollado aún.
Búsqueda y filtrado avanzado	Pendiente	No desarrollado aún.
Hardening de producción	Pendiente	Secretos, HTTPS y rate limiting pendientes.

1.3 Arquitectura Implementada

La arquitectura del sistema está compuesta por tres contenedores que conviven en la red interna mercadoliebre-net, gestionada por Docker Compose. Cada contenedor tiene una responsabilidad clara: Nginx sirve la interfaz web y actúa de intermediario hacia la API,

Node.js contiene toda la lógica del negocio y MySQL almacena los datos. Esta separación permite que cada parte escale o se reemplace de forma independiente.



Servicios definidos

Servicio (Compose)	Contenedor	Tecnología	Puerto host → contenedor	Función
frontend	mercadoliebre_frontend	Nginx + estáticos Vite	8080 → 80	Sirve la SPA y redirige /api/ al backend.
api-service	mercadoliebre_api	Node.js (Express)	Solo red interna (3000)	API REST, lógica de negocio, subida a Cloudinary.
db-service	mercadoliebre_db	MySQL 8.0	3306 → 3306	Persistencia de datos; esquema inicial en init-db/init.sql.

Relaciones entre servicios

- El usuario accede a la interfaz en http://localhost:8080. Nginx recibe la petición.
- Si la petición comienza con /api/, Nginx la redirige a api-service:3000 usando el nombre DNS interno de Docker (no localhost).
- La API se conecta a MySQL con el host db-service (mismo principio de nombre de servicio).
- api-service espera a que db-service esté disponible antes de arrancar (depends_on + healthcheck).
- La base de datos no está expuesta directamente al exterior desde la perspectiva de la red interna.

Nota: el frontend compilado vive en el contenedor de Nginx; Node.js solo sirve la API. Esto cumple explícitamente el requisito de tener más de un servicio y de que exista comunicación HTTP entre ellos (Nginx → API).

1.4 Comunicación entre Servicios y Endpoints

Resolución por nombres (Docker)

Dentro de la red de Docker Compose, los contenedores se identifican por el nombre de su servicio en lugar de por una dirección IP. Esto evita problemas cuando Docker reasigna IPs al reiniciar los contenedores.

```
# Desde Nginx:
proxy_pass http://api-service:3000/api/;

# Desde Node.js:
DB_HOST=db-service
```

Endpoints de autenticación

Método	Ruta	Descripción
POST	/api/auth/register	Crea un nuevo usuario. Campos: email, password, nombre, apellido.
POST	/api/auth/login	Inicia sesión. Devuelve un token JWT y los datos del usuario.
GET	/api/auth/me	Devuelve el perfil del usuario autenticado. Requiere Bearer token.

Ejemplo — Registro (201 Created)

```
// Solicitud:
{
  "email": "ana@ejemplo.com",
  "password": "MiClaveSegura1",
  "nombre": "Ana",
  "apellido": "Torres"
}

// Respuesta exitosa:
{
  "token": "<jwt>",
  "user": {
    "id": "<uuid>",
    "email": "ana@ejemplo.com",
    "nombre": "Ana",
    "apellido": "Torres"
  }
}
```

Ejemplo — Login (200 OK / 401 Error)

```
// Solicitud:
{ "email": "ana@ejemplo.com", "password": "MiClaveSegura1" }

// Respuesta exitosa (200):
{ "token": "<jwt>", "user": { ... } }

// Respuesta fallida (401):
{ "error": "Credenciales inválidas" }
```

Endpoints de productos

Método	Ruta	Autenticación	Descripción
GET	/api/productos	No	Lista todos los productos. Acepta filtros ?tienda_id= y ?activo=.
POST	/api/productos	Bearer JWT	Crea un nuevo producto (incluye tienda_id).
PATCH	/api/productos/:id	Bearer JWT	Actualiza un producto existente.
DELETE	/api/productos/:id	Bearer JWT	Elimina un producto.

Endpoints de tiendas y otros

Método	Ruta	Descripción
GET	/api/tiendas	Lista de todas las tiendas.
GET	/api/tiendas/:id/vista-publica	Devuelve tienda, tema y productos en un solo objeto para la vitrina pública.
GET	/api/tiendas/mias	Tienda del usuario autenticado (requiere JWT).
POST/PATCH	/api/tiendas, /api/tiendas/:id	Crear o actualizar tienda (requiere JWT).
GET	/api/categorias?tienda_id=...	Categorías de una tienda.
GET/POST/PATCH	/api/temas, /api/temas/:id	Gestión del tema visual (escritura requiere JWT).
POST	/api/media/upload	Sube una imagen a Cloudinary (requiere JWT).
GET	/api/health	Diagnóstico rápido del estado de la API.

Importante: las rutas /api/articulos, /api/registro y /api/login mencionadas en versiones anteriores del informe no existen en el código actual. Los endpoints correctos son los que se listan en estas tablas.

1.5 Modelo de Datos (MySQL)

El esquema de la base de datos se crea automáticamente al iniciar el contenedor db-service, a partir del archivo init-db/init.sql. Las imágenes no se guardan en el disco del servidor — solo se almacena la URL de Cloudinary donde está alojada cada imagen.

Tabla	Campos principales
usuarios	id (UUID), email, nombre, apellido, password_hash, creado_en
tiendas	id, usuario_id, datos comerciales, imagen_logo_url, imagen_banner_url, dias_abierto (JSON)
temas	id, tienda_id, colores, fuentes, estilo_plantilla
categorias	id, tienda_id, nombre, icono, orden
productos	id (auto_increment), tienda_id, nombre, descripcion, precio, imagen_url, categoria, disponible, activo

1.6 Configuración del Entorno

El sistema usa variables de entorno para separar la configuración del código fuente. Esto permite cambiar credenciales, puertos o nombres de servicios sin modificar el código, y facilita la transición entre los entornos de desarrollo y producción.

Variable	Datos	Servicio	Descripción
DB_HOST	NN	api-service	Hostname de MySQL en Docker: db-service.
MYSQL_USER	admin	api-service	Usuario de la base de datos.
MYSQL_PASSWORD	rootpassword	api-service	Contraseña de la base de datos.
MYSQL_DATABASE	mercadoliebre_db_prond	api-service	Nombre de la base de datos: mercadoliebre_db_prod.
JWT_SECRET	NN	api-service	Clave para firmar los tokens JWT.
CLOUDINARY_CLOUD_NAME	NN	api-service	Nombre de la cuenta en Cloudinary.
CLOUDINARY_API_KEY	NN	api-service	Clave de acceso a Cloudinary.
CLOUDINARY_API_SECRET	NN	api-service	Secreto de la cuenta Cloudinary.
MYSQL_ROOT_PASSWORD	NN	db-service	Contraseña del usuario root de MySQL.

Puertos y acceso

- Interfaz web: http://localhost:8080 (mapeo 8080:80 del contenedor frontend).
- API REST (mismo origen): http://localhost:8080/api/... (navegador y Postman usan la misma URL base).
- MySQL desde el host (opcional): localhost:3306.
- Desarrollo local sin Docker (solo referencia): Vite suele usar el puerto 5173 con proxy de /api hacia http://127.0.0.1:3000.

Volumen de datos

El volumen mysqldata se monta en /var/lib/mysql del contenedor db-service. Esto garantiza que los datos persistan aunque el contenedor se reinicie o se elimine. El motor es MySQL — las variables tipo POSTGRES_* no aplican a este proyecto.

2.Justificación Técnica

Esta sección resume las razones detrás de las principales decisiones de diseño, orientadas a cumplir los requisitos de la materia de Sistemas Distribuidos y a mantener el proyecto mantenible a lo largo del semestre.

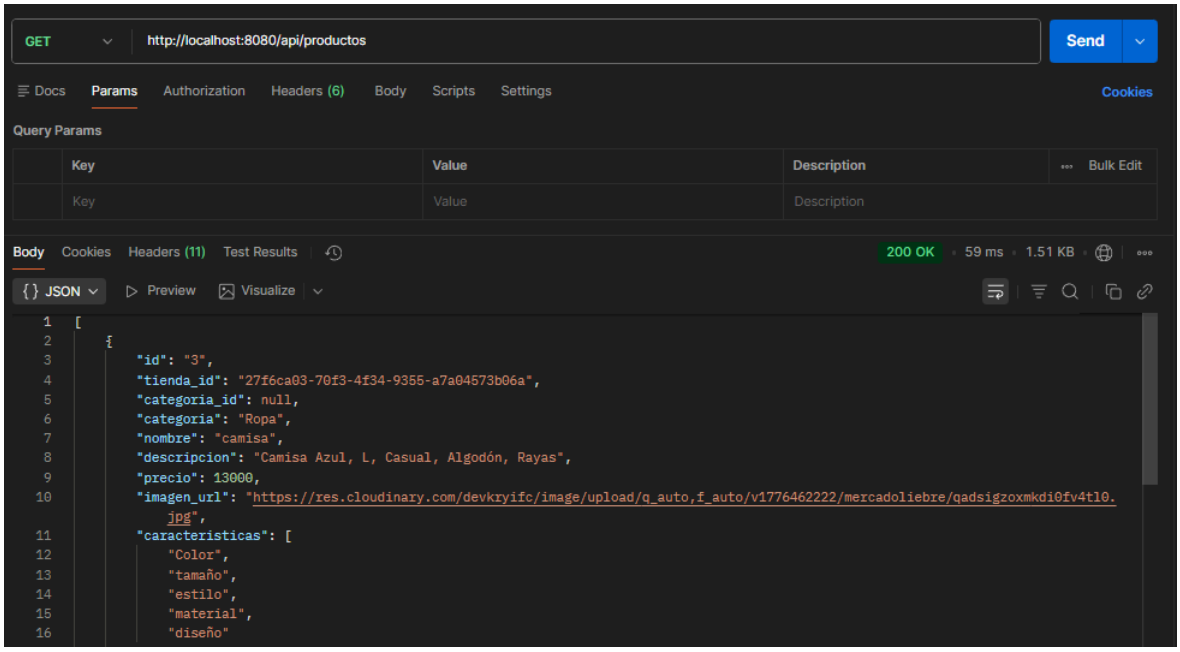
Decisión	Justificación
Docker Compose	Permite que todo el equipo levante el entorno completo con un solo comando. Elimina el clásico problema de 'en mi máquina sí funciona'.
MySQL en contenedor	Los datos persisten entre reinicios gracias al volumen mysqldata. El modelo relacional refleja claramente las relaciones entre usuarios, tiendas y productos.
Nginx como reverse proxy	Unifica el origen de la SPA y la API bajo el mismo puerto (8080), lo que simplifica CORS y demuestra explícitamente la comunicación HTTP entre servicios.
JWT para sesiones	No requiere almacenar el estado de la sesión en el servidor. El frontend incluye el token en cada petición protegida mediante el encabezado Authorization.
Cloudinary para imágenes	Las imágenes no ocupan espacio en el contenedor; solo se guarda la URL. Esto hace el sistema más ligero y fácil de desplegar.

Evidencias Requeridas

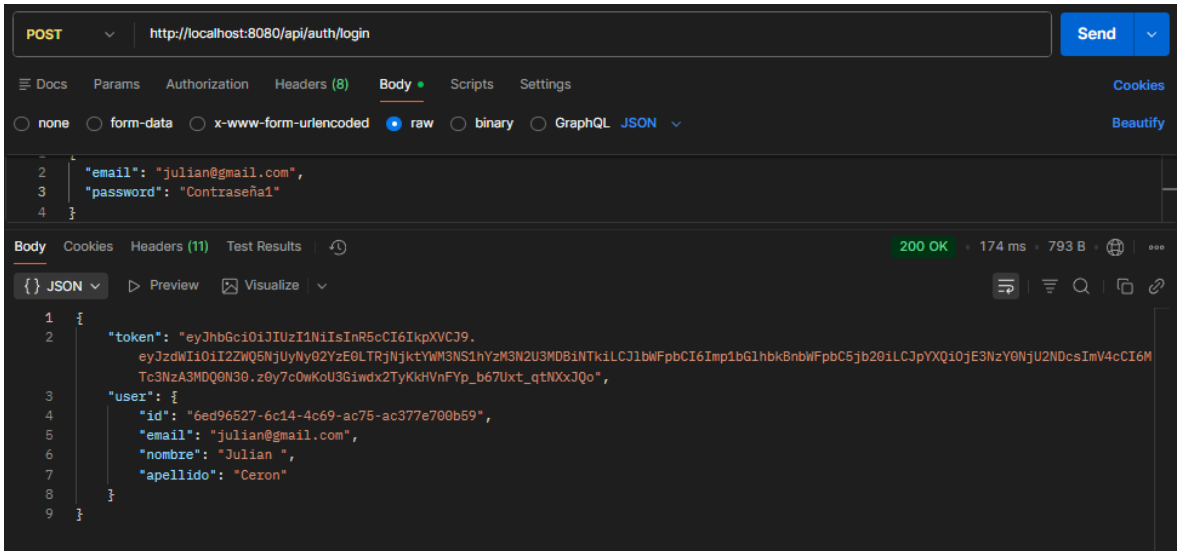
1. **Servicios en ejecución:** Captura del comando docker ps en la terminal o de la vista de contenedores en Docker Desktop, mostrando los contenedores backend y mercado-liebre-db activos con sus puertos y estado "Up".

```
(base) PS C:\Users\julia\OneDrive\Documentos\Mercado_Liebre> Docker ps
CONTAINER ID   IMAGE                        COMMAND                  CREATED          S
TATUS                    PORTS                                NAMES
565f04ebc0e3   mercado_liebre-frontend      "/docker-entrypoint.…"   15 minutes ago   U
p 15 minutes             0.0.0.0:8080->80/tcp                 mercadoliebre_frontend
b7a96fcdb746   mercado_liebre-api-service   "docker-entrypoint.s…"   15 minutes ago   U
p 15 minutes             3000/tcp                             mercadoliebre_api
6d9807fafe69   mysql:8.0                    "docker-entrypoint.s…"   15 minutes ago   U
p 15 minutes (healthy)   0.0.0.0:3306->3306/tcp, 33060/tcp    mercadoliebre_db
(base) PS C:\Users\julia\OneDrive\Documentos\Mercado_Liebre>
Ctrl+K to generate command
```

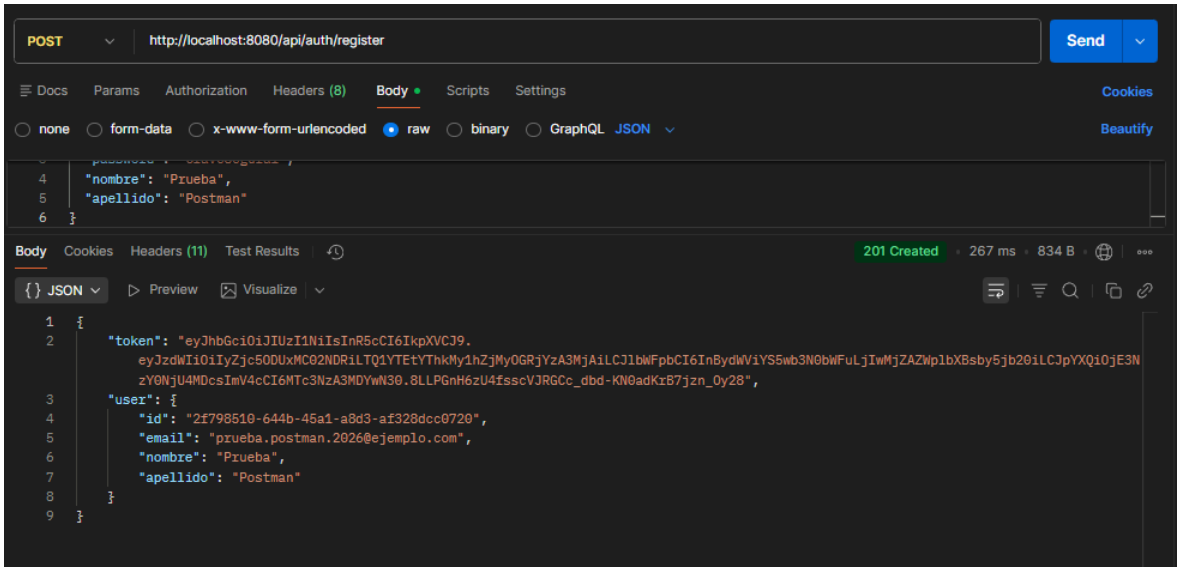
2. **Prueba del endpoint GET /api/productos:** Respuesta del endpoint GET localhost:8080/api/productos con datos reales de productos en formato JSON.



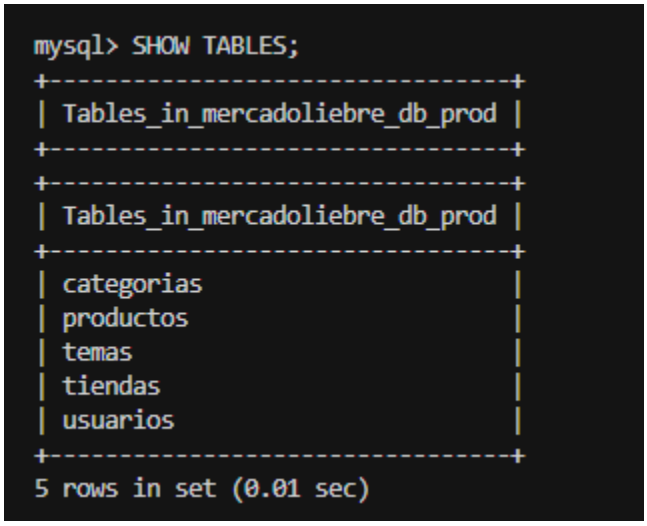
3. **Prueba del endpoint POST /api/auth/login:** Captura mostrando una solicitud POST exitosa con el cuerpo de credenciales enviado y la respuesta del servidor (código 200 OK).



4. **Prueba del endpoint POST /api/register:** Captura mostrando el registro de un nuevo usuario con respuesta 201 Created.



5. **Datos en la base de datos:** Captura desde un cliente de base de datos (DBeaver, pgAdmin, TablePlus o psql) mostrando las tablas usuarios, temas, artículos y categorías con registros almacenados.



```
mysql> SHOW TABLES;
+-------------------------------+
| Tables_in_mercadoliebre_db_prod |
+-------------------------------+
+-------------------------------+
| Tables_in_mercadoliebre_db_prod |
+-------------------------------+
| categorias                    |
| productos                     |
| temas                         |
| tiendas                       |
| usuarios                      |
+-------------------------------+
5 rows in set (0.01 sec)
```